Data Structures and Algorithms

Scribe submitted for 27th October to

Indian Statistical Institute

by

Sayak Basak	CS2632
Pritam Das	CS2627
Srijan Saha	CS2635
Anik Barury	CS2604
Shubha Deep Chatterjee	CrS2615
Soumyadeep Naskar	CS2634



DEPARTMENT OF COMPUTER SCIENCE
INDIAN STATISTICAL INSTITUTE
203 BARRACKPORE TRUNK ROAD, KOLKATA - 700108
WEST BENGAL, INDIA

October, 2026

1. Introduction

An AVL tree is a self-balancing binary search tree (BST). It was invented by Landis and Adelson-Velsky. Like an ordinary BST it stores smaller keys in the left subtree and larger keys in the right subtree. It also keeps a balance condition for each node so that the tree does not get too tall.

2. Properties of AVL Trees

An AVL tree has the following two properties:

- Binary Search Tree property: for each node x the keys in the left subtree are smaller than x and the keys in the right subtree are larger than x (assuming distinct keys).
- AVL balancing property: The height difference between the left and right subtrees of any node is no more than one.

$$|\text{height(left subtree)} - \text{height(right subtree)}| \leq 1$$

The balance factor of a node x is defined to be:

$$\text{BF}(x) = \text{height(left subtree)} - \text{height(right subtree)}$$

An AVL tree is a balanced tree where each node has a balance factor of -1, 0, or $+1$.

3. Height of AVL Tree

AVL trees are approximately balanced, so the height is logarithmic in the number of nodes. A typical upper bound is $h = O(\log n)$, where n is the number of nodes and h is the height. More precisely, the height is at most about $1.44 \log_2(n + 2)$ up to a small additive constant depending on the height convention.

To determine the minimum number of nodes in an AVL tree of height h, consider the AVL tree of height h that is the sparsest. One subtree must have height $h - 1$ and the other may have height $h - 2$ and still satisfy the AVL balance condition:

$$N_h = N_{h-1} + N_{h-2} + 1$$

Here, N_h is the minimum number of nodes in an AVL tree of height h. This recurrence is closely related to the Fibonacci recurrence and accounts for why the height of an AVL tree is $O(\log n)$. (The exact starting values depend on whether an empty tree is considered to have height -1 or 0.)

4. Insertion in AVL Tree

Insertion is done in steps:

1. Insert new key just as in a normal binary search tree.
2. Starting from the inserted node, move up to the root, updating subtree heights or balance factors.
3. Find the lowest ancestor whose balance factor is $+2$ or -2.
4. Restore the AVL property by performing the appropriate rotation(s).
5. (If required) check ancestors and tree should satisfy AVL property at each and every node

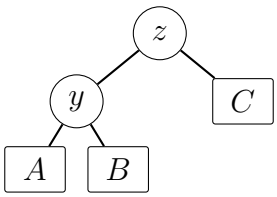
5. Using Rotation to Reestablish Your Balance

A rotation changes the local structure of a tree, but keeps the BST ordering of keys. The four cases of imbalance are identified by the path from the unbalanced node to the newly inserted key.

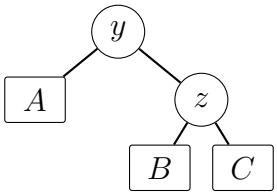
5.1 LL Case – Right Rotation

The LL case arises when a node becomes left-heavy due to insertion in the left subtree of its left child. One right rotation brings the balance back.

Before rotation:



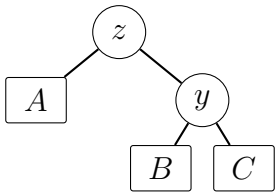
After a right rotation at z:



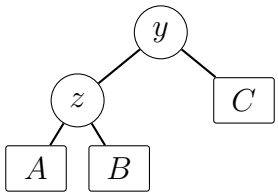
5.2 RR Case – Left Rotation

The RR case occurs when a node becomes right-heavy because insertion took place in the right subtree of its right child. A single left rotation restores balance.

Before rotation:



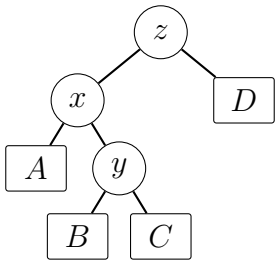
After a left rotation at z:



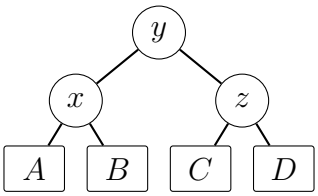
5.3 LR Case – Left Rotation, then Right Rotation

The LR case occurs when insertion takes place in the right subtree of the left child of the unbalanced node. First perform a left rotation at the left child; then perform a right rotation at the unbalanced node.

Before rotations:



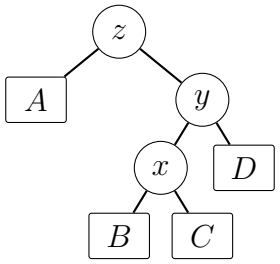
After rotations:



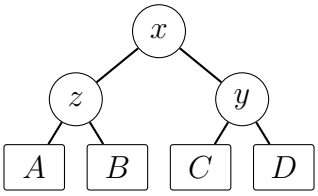
5.4 RL Case – Right Rotation, then Left Rotation

The RL case occurs when insertion takes place in the left subtree of the right child of the unbalanced node. First perform a right rotation at the right child; then perform a left rotation at the unbalanced node.

Before rotations:

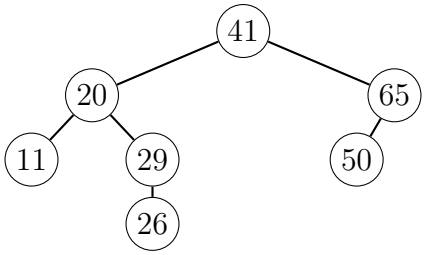


After rotations:



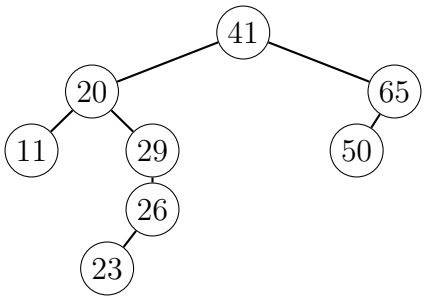
6. Worked Example 1: Insert 23

Consider the following AVL tree. The keys are arranged according to the BST property, and the tree is initially balanced.



Step 1: Insert 23 as in a BST. The search path is $41 \to 20 \to 29 \to 26$. Since 23 is smaller than 26, it becomes the left child of 26.

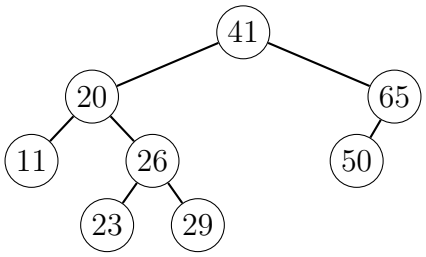
After inserting 23:



Step 2: Node 29 is the first (lowest) unbalanced ancestor. Its left subtree is two levels taller than its right subtree, and the insertion occurred in the left subtree of its left child. This is an LL case.

Step 3: Perform a right rotation at node 29.

After the right rotation:

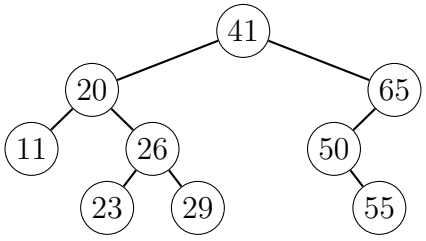


The BST ordering is preserved and the AVL balance condition is restored. The resulting tree is balanced.

7. Worked Example 2: Insert 55

Start with the balanced tree obtained after inserting 23. Insert 55 using the ordinary BST procedure. The search path is $41 \to 65 \to 50$. Since 55 is greater than 50 but less than 65, it becomes the right child of 50.

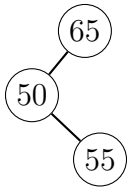
After inserting 55:



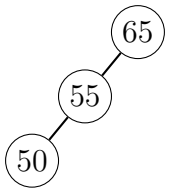
The subtree rooted at 65 becomes left-heavy, but its left child 50 is right-heavy. Therefore, this is an LR case relative to node 65 (left child, then right).

Step 1: Left-rotate at node 50.

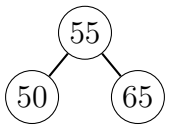
Before:



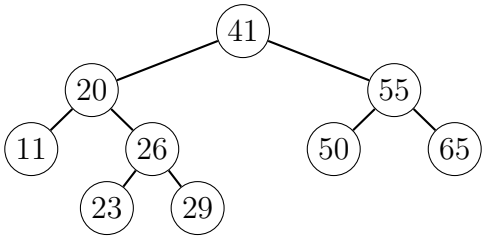
After the left rotation:



Step 2: Right-rotate at node 65. The resulting right subtree is:



The complete AVL tree after rebalancing is:



The tree again satisfies both the BST property and the AVL balance condition.

8. Complexity Analysis

The height of an AVL tree is $O(\log n)$, so the worst case time complexity of any of search, insert or delete operations is $O(\log n)$. An insertion does at most a constant number of rotations at the first unbalanced ancestor. (height / balance updates may be required on the path to the root).

Operation	Worst-case time
Search	$O(\log n)$
Insertion	$O(\log n)$
Deletion	$O(\log n)$